5

웹 서비스와 GPT 연동

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

웹 서비스와 GPT 연동

- **웹 서비스와 인공지능**

지금까지 우리는 Python, 데이터 처리, 그리고 기본적인 프로그램을 통해 데이터를 분석하고 결과를 출력하는 방법을 학습했습니다. 하지만 실제 환경에서는 단순한 프로그램을 넘어, 사용자가 직접 사용할 수 있는 "웹 서비스 형태"로 구현하는 것이 중요합니다.

웹 서비스는 인터넷을 통해 누구나 접근할 수 있으며, 사용자의 입력을 받아 서버에서 처리한 뒤 결과를 다시 전달하는 구조로 동작합니다.

- **웹 서비스와 인공지능(GPT)의 활용**

최근에는 인공지능 기술의 발전으로, 웹 서비스에 GPT와 같은 AI 모델을 결합하여 사용자와 대화하고 문제를 해결하는 서비스가 증가하고 있습니다. 기존의 웹 서비스가 정해진 기능을 수행하는 데 집중했다면, GPT를 활용한 웹 서비스는 사용자의 질문을 이해하고 상황에 맞는 답변을 생성하는 지능형 서비스로 발전하고 있습니다.

GPT는 사용자의 자연어 입력을 이해하고 그 의미를 분석하여 적절한 답변을 생성하는 인공지능 모델입니다. 사용자가 문장 형태로 질문을 입력하면, GPT는 질문의 의도를 파악하고 관련 정보를 바탕으로 자연스럽게 답변을 제공합니다. 이러한 특징 덕분에 다양한 형태의 질문에도 유연하게 대응할 수 있습니다.

웹 서비스에서 GPT는 사용자의 질문을 해석하고 답변을 생성하는 역할을 수행하며, 이전 대화 내용을 바탕으로 맥락을 유지하는 대화형 기능도 제공합니다. 이를 통해 단순한 정보 제공을 넘어 사용자와 상호작용하는 서비스 구현이 가능합니다.

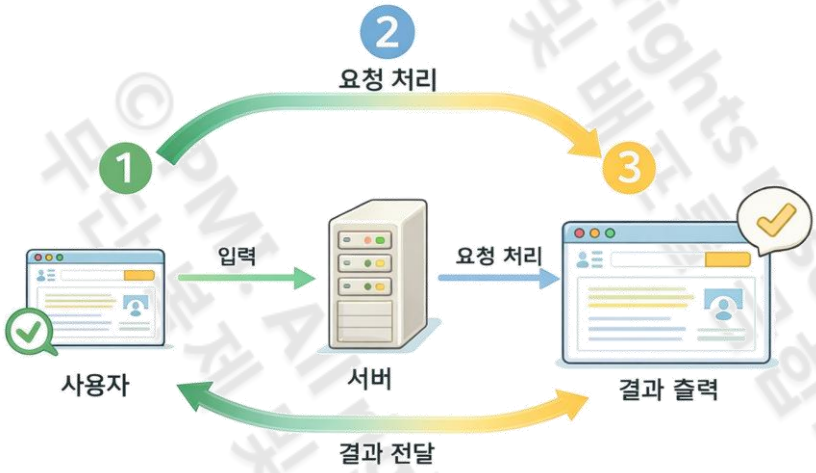
이러한 GPT 기반 웹 서비스는 교육, 의료, 고객 상담 등 다양한 분야에서 활용되고 있으며, 사용자는 복잡한 기능을 탐색하지 않고도 질문을 입력하는 것만으로 필요한 정보를 얻을 수 있습니다.

- **웹 서비스 동작 구조**

웹 서비스는 사용자의 입력을 받아 서버에서 처리한 뒤 결과를 다시 전달하는 클라이언트-서버 구조로 동작합니다.

먼저 사용자가 웹 브라우저에서 버튼을 클릭하거나 정보를 입력하면, 해당 데이터는

JavaScript를 통해 서버로 전송됩니다. 서버는 전달받은 요청을 분석하고 필요한 작업을 수행하며, 경우에 따라 데이터베이스 조회나 외부 API 호출을 함께 진행합니다.



이후 서버는 처리 결과를 다시 클라이언트로 반환하고, 웹페이지는 이를 화면에 출력하여 사용자에게 보여줍니다. 이러한 과정은 매우 빠르게 반복되며, 사용자는 실시간으로 결과를 확인할 수 있습니다.

이와 같은 구조를 통해 웹 서비스는 별도의 프로그램 설치 없이도 다양한 기능을 제공할 수 있으며, 사용자와 서버 간의 지속적인 상호작용이 가능해집니다.

이와 같은 웹 서비스의 동작 구조를 이해하면, 사용자의 입력이 어떻게 서버로 전달되고 처리되는지, 그리고 결과가 어떤 과정을 통해 다시 사용자에게 전달되는지를 알 수 있습니다.

이제 이러한 구조를 바탕으로, 실제로 웹페이지를 만들고 사용자의 입력을 처리하는 과정을 직접 구현해 보겠습니다.

다음 절에서는 VS Code를 이용하여 웹페이지를 구성하고 실행하는 방법을 단계별로 학습합니다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

웹페이지 제작 및 실행하기

- **VS Code 설치 및 초기 설정하기**

지금까지 우리는 Google Colab과 같은 환경을 사용하여 Python 코드 작성과 실행을 진행해 왔습니다. Colab은 별도의 설치 없이 사용할 수 있어 매우 편리하지만, 웹페이지를 제작하는 과정에서는 HTML, CSS, JavaScript 파일을 직접 관리하고 여러 파일을 함께 구성해야 하기 때문에 보다 전문적인 개발 환경이 필요합니다

이러한 이유로 이번 장에서는 VS Code(Visual Studio Code)를 사용하여 웹페이지를 제작하고 실행하는 방법을 학습합니다. VS Code는 다양한 파일을 한 번에 관리할 수 있고, 웹 개발에 필요한 기능을 확장 프로그램을 통해 쉽게 추가할 수 있어 웹 서비스 개발에 매우 적합한 도구입니다.

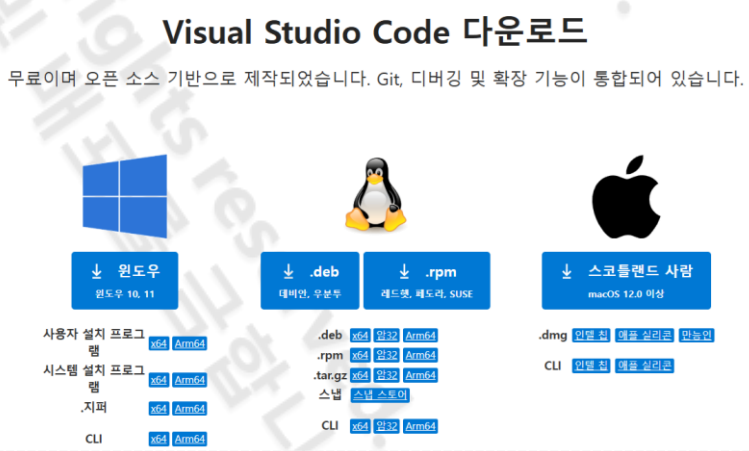
이번 절에서는 VS Code를 설치하고, 웹페이지를 보다 편리하게 개발할 수 있도록 기본 환경을 설정합니다.

1) VS Code 설치하기

VS Code는 무료로 제공되는 코드 편집기로, HTML, CSS, JavaScript와 같은 웹 개발 언어를 효율적으로 작성할 수 있는 도구입니다.

다음과 같은 순서로 설치를 진행합니다.

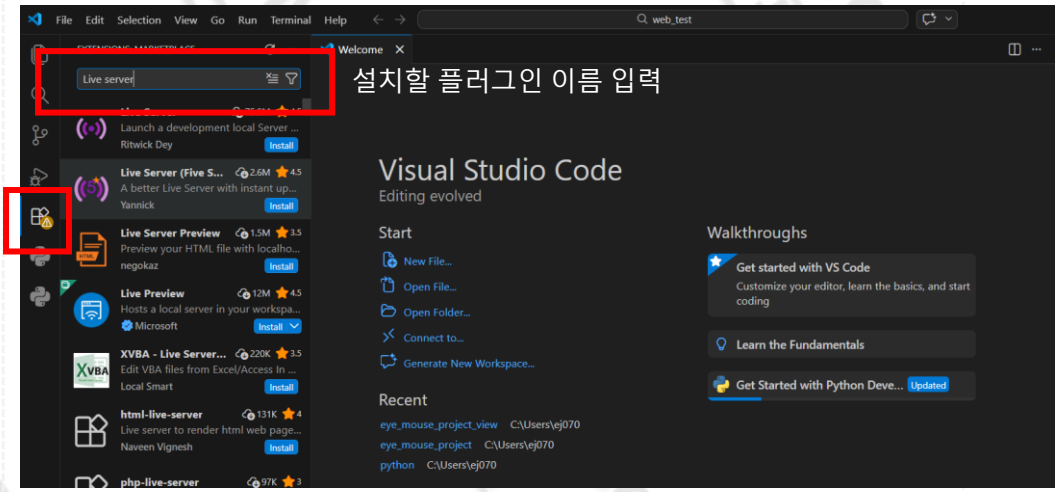
1. VS Code 공식 사이트(https://code.visualstudio.com/)에 접속합니다.
2. 자신의 운영체제에 맞는 설치 파일을 다운로드합니다.(Windows, macOS, Linux 지원)



2) 추천 플러그인 설치하기

VS Code를 설치한 후에는 웹 개발을 더욱 편리하게 하기 위해확장 기능(플러그인)을 추가로 설치합니다.

VS Code 실행 후, 좌측 메뉴의 확장 기능(Extensions) 아이콘을 클릭하여 다음 플러그인을 설치합니다.



설치할 플러그인 이름	설명
Live Server	HTML 파일을 브라우저에서 실시간으로 실행 및 미리보기
Prettier	코드 자동 정렬 (선택)
HTML CSS Support	HTML, CSS 문법 강조
JavaScript Language Basics	JavaScript 문법 지원 (기본 제공)

- 플러그인을 사용하는 이유

이러한 플러그인을 설치하면 다음과 같은 장점이 있습니다

- Live Server를 통해 웹페이지를 서버 형태로 실행할 수 있습니다.
- 코드 작성 시 자동 정렬과 문법 강조를 통해 가독성이 향상됩니다.

- 오류를 줄이고 개발 속도를 높일 수 있습니다.

특히 Live Server는 웹페이지를 수정할 때마다 자동으로 새로 고침 되므로, 변경 사항을 즉시 확인할 수 있어 매우 중요한 도구입니다.

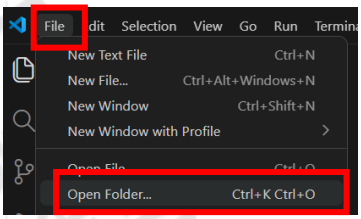
3) 새 프로젝트 폴더 만들기

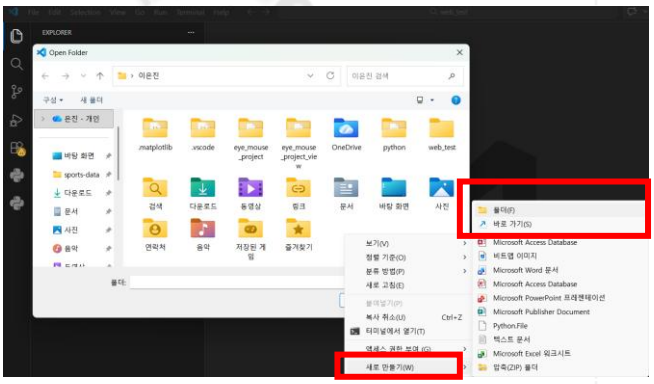
웹페이지를 제작할 때는 관련된 파일들을 하나의 폴더 안에서 관리하는 것이 중요합니다. 이를 통해 파일 간 연결을 쉽게 하고, 프로젝트 구조를 체계적으로 유지할 수 있습니다.

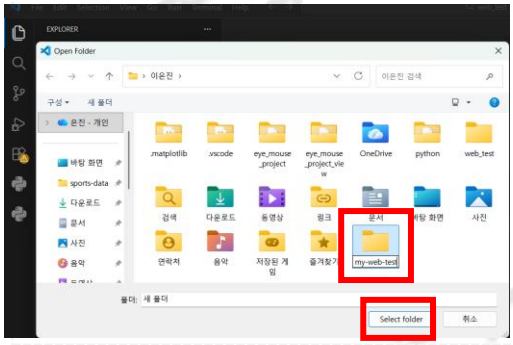
다음과 같은 순서로 새 프로젝트 폴더를 생성합니다.

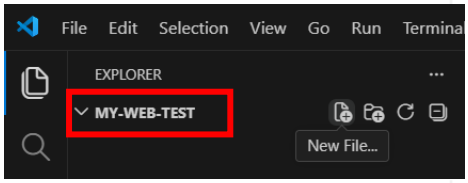
1. 원하는 위치에 새 폴더를 생성합니다. (예: my-web-test).
2. VS Code에서 상단 메뉴의 파일 → 폴더 열기를 선택한 후, 생성한 my-web-test 폴더를 열어줍니다.

이 과정을 통해 VS Code에서 해당 폴더를 하나의 프로젝트로 인식하게 됩니다.





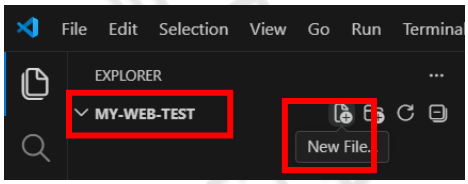


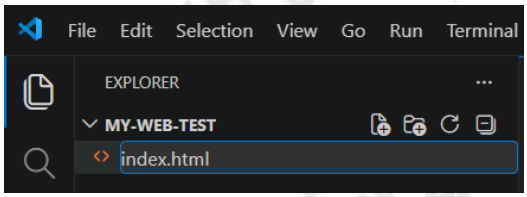


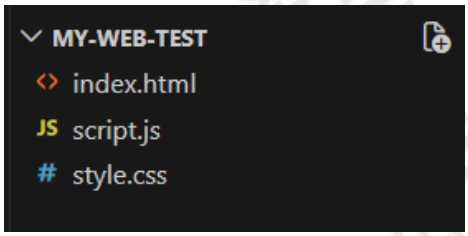
4) 기본 웹 파일 만들기

웹페이지는 하나의 파일로 구성되는 것이 아니라, 각 역할을 담당하는 여러 파일이 함께 동작하여 만들어집니다.

프로젝트 폴더 안에 다음과 같은 기본 파일 3개를 생성합니다.







- 각 파일의 역할

  - index.html : 웹페이지의 구조를 담당하는 파일로, 화면에 표시되는 내용을 정의합니다.
  - style.css : 웹페이지의 디자인을 담당하며, 색상, 글꼴, 여백 등을 설정합니다.
  - script.js : 웹페이지의 동작을 담당하며, 버튼 클릭이나 사용자 입력과 같은 기능을 처리합니다.

- 파일을 나누어 작업하는 이유

웹 개발에서는 기능을 분리하여 관리하는 것이 매우 중요합니다.

  - 구조(HTML), 디자인(CSS), 동작(JavaScript)을 나누면 코드가 정리됩니다.
  - 유지보수와 수정이 쉬워집니다.
  - 실제 웹 서비스 개발에서도 동일한 구조를 사용합니다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

5) 웹페이지 코드 작성하기

앞에서 생성한 index.html, style.css, script.js 파일에 실제 코드를 입력하여 웹페이지를 완성합니다.

index.html 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 index.html 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>첫 웹페이지</title>

  <!-- CSS 연결 -->
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <h1>안녕하세요</h1>
  <p>ChatGPT가 알려준 웹 예제입니다.</p>

  <!-- 버튼 클릭 시 JS 함수 실행 -->
  <button onclick="sayHello()">눌러보세요</button>

  <!-- JS 연결 -->
  <script src="script.js"></script>
</body>
</html>
```

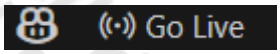
© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

웹페이지 제작 및 실행하기

- 코드 설명

- <!DOCTYPE html> : 현재 문서가 HTML5 형식임을 선언합니다.
- <html lang="ko"> : HTML 문서의 시작이며, 언어를 한국어로 설정합니다.
- <head> : 웹페이지의 설정 정보를 담는 영역입니다. (화면에는 보이지 않음).
- <meta charset="UTF-8"> : 한글이 깨지지 않도록 문자 인코딩을 설정합니다.
- <title>첫 웹페이지</title> : 브라우저 탭에 표시되는 제목입니다.
- <link rel="stylesheet" href="style.css"> : CSS 파일을 연결하여 디자인을 적용합니다.
- <body> : 사용자에게 실제로 보이는 내용을 작성하는 영역입니다.
- <h1>안녕하세요</h1> : 가장 큰 제목을 표시합니다.
- ... : 일반 문장을 출력하는 태그입니다.
- <button onclick="sayHello( )"> : 버튼 클릭 시 sayHello() 함수가 실행됩니다.
- <script src="script.js"></script> : JavaScript 파일을 연결하여 동작을 구현합니다.

웹페이지 제작 및 실행하기

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭 

style.css 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 style.css 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```
body {
  font-family: '맑은 고딕', sans-serif;
  background-color: #f5f5f5;
  padding: 30px;
}

h1 {
  color: #007acc;
}
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

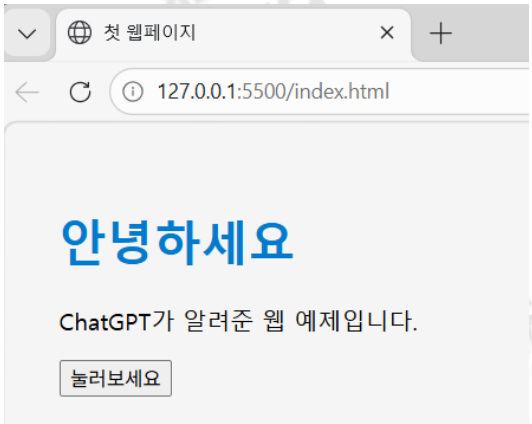
웹페이지 제작 및 실행하기

- 코드 설명

- body { ... } : 웹페이지 전체 영역에 스타일을 적용합니다.
- font-family: '맑은 고딕', sans-serif; : HTML 문서의 시작이며, 언어를 한국어로 설정합니다.
- background-color: #f5f5f5; : 배경색을 밝은 회색으로 설정합니다.
- padding: 30px; : 화면 가장자리와 내용 사이에 여백을 줍니다.
- h1 { color: #007acc; } : 제목(h1)의 글자 색을 파란색으로 변경합니다.

- 코드 실행 결과

  - Vs code 오른쪽 하단에 Go Live 클릭



style.css 코드 작성

- 코드 작성

```
function sayHello() {
  alert("Hello! ChatGPT 웹 예제입니다.");
}
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

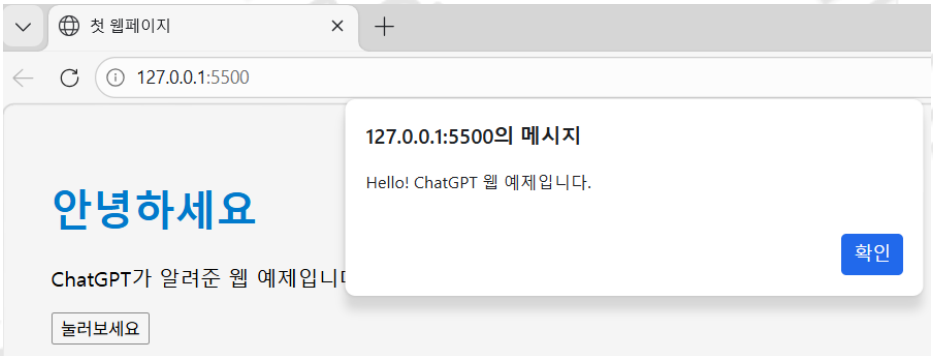
웹페이지 제작 및 실행하기

- 코드 설명

- function sayHello() { ... } : sayHello라는 이름의 함수를 정의합니다. 이 함수는 버튼 클릭 시 실행됩니다.
- alert("Hello! ChatGPT 웹 예제입니다."); : 알림창(alert)을 띄워 메시지를 화면에 출력합니다.

- 코드 실행 결과

  - Vs code 오른쪽 하단에 Go Live 클릭
  - "눌러보세요"를 클릭



© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

이번 단계에서는 단순히 하나의 웹페이지를 보여주는 것을 넘어서, 여러 페이지를 서로 연결하고 사용자 인증 기능을 추가하여 웹 서비스를 실제 서비스 형태에 가깝게 확장합니다. 이를 위해 첫 화면, 로그인 화면, 그리고 이동용 페이지를 각각 구성하고, 버튼을 클릭했을 때 원하는 페이지로 이동할 수 있도록 합니다. 또한 login.html 페이지에서는 Firebase를 활용하여 회원가입과 로그인 기능을 구현합니다.

앞에서 만든 웹페이지는 하나의 화면만 동작하는 가장 기본적인 예제였습니다. 하지만 실제 웹 서비스는 여러 개의 페이지가 서로 연결되어 있으며, 사용자의 선택에 따라 다른 화면으로 이동할 수 있어야 합니다. 또한 단순히 정보를 제공하는 것을 넘어, 회원가입과 로그인 기능을 통해 사용자를 구분하고 서비스 기능을 다르게 제공하는 경우가 많습니다.

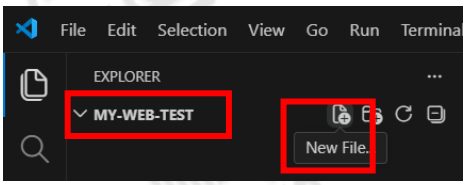
따라서 이번 단계는 정적인 웹페이지에서 더 나아가 페이지 이동과 사용자 인증 기능이 있는 웹 서비스 구조를 이해하는 중요한 과정입니다.

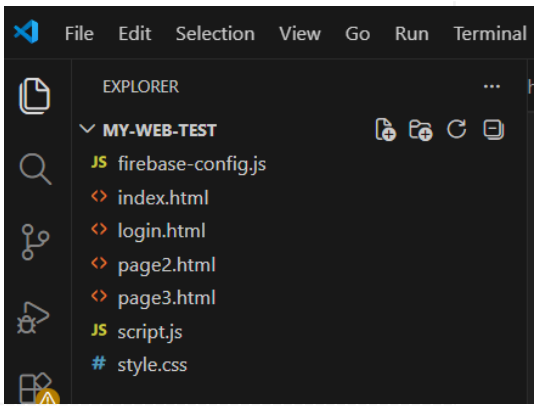
- 프로젝트 구조 이해하기

  여러 페이지와 사용자 인증 기능이 추가되면 웹 서비스는 하나의 파일이 아닌 여러 파일이 서로 연결된 구조로 동작하게 됩니다. 이때 각 파일은 서로 다른 역할을 수행하며, 이들이 유기적으로 연결되어 하나의 서비스로 동작하게 됩니다.

  따라서 이러한 기능을 올바르게 구현하기 위해서는 각 파일의 역할을 이해하고, 프로젝트를 체계적으로 구성하는 것이 중요합니다.

  다음과 같은 구조로 프로젝트를 구성합니다.





© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

프로젝트 구조 설명

- index.html : 웹사이트의 첫 화면(메인 페이지)
- login.html : 회원가입 및 로그인 기능을 수행하는 페이지
- page2.html, page3.html : 버튼 클릭 시 이동하는 페이지
- style.css : 전체 페이지 디자인
- script.js : 공통 JavaScript 기능
- firebase-config.js : Firebase와 웹페이지를 연결하는 설정 파일

index.html 코드 작성

1. 작업 방법

- VS Code 왼쪽 탐색기에서 index.html 클릭
- 기존 내용이 있다면 모두 삭제
- 아래 코드를 작성 또는 Github 복사

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>메인 페이지</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>메인 페이지</h1>
  <button onclick="location.href='login.html'">
    회원가입 / 로그인
  </button>

  <button onclick="location.href='page2.html'">
    페이지 2
  </button>
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

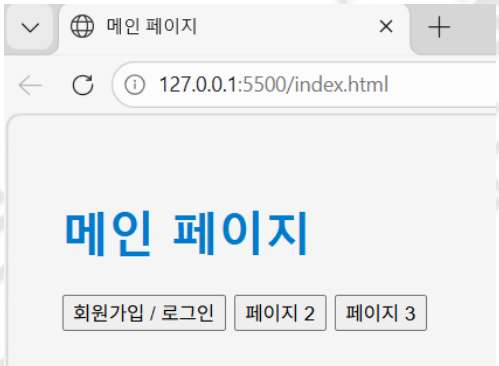
```
    <button onclick="location.href='page3.html'">
      페이지 3
    </button>

  </body>
  </html>
```

- 코드 설명

- <button onclick="location.href='login.html'"> : 버튼 클릭 시 login.html 페이지로 이동
- <button onclick="location.href='page2.html'"> : 버튼 클릭 시 page2.html로 이동
- <button onclick="location.href='page3.html'"> : 버튼 클릭 시 page3.html로 이동

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭



© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- Firebase 프로젝트 생성 및 설정하기

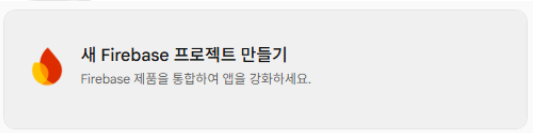
웹페이지에서 회원가입과 로그인 기능을 구현하기 위해서는 사용자의 정보를 저장하고 인증을 처리할 수 있는 시스템이 필요합니다.

하지만 이러한 기능을 직접 서버로 구현하는 것은 복잡하고 시간이 많이 소요되기 때문에, 이번 실습에서는 Firebase를 활용하여 간단하게 사용자 인증 기능을 구현합니다. Firebase는 별도의 서버 구축 없이도 회원가입과 로그인 기능을 제공하는 클라우드 기반 서비스로, 웹 애플리케이션에서 쉽게 사용할 수 있습니다. 이제 Firebase를 사용하기 위한 프로젝트를 생성하고,웹페이지와 연결하는 과정을 단계별로 진행해 보겠습니다.

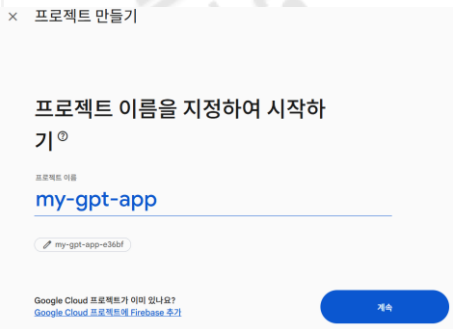
Firebase 프로젝트 만들기

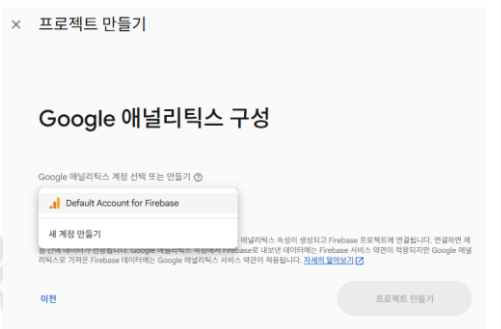
Firebase를 사용하기 위해서는 먼저 웹 애플리케이션을 관리할 수 있는 프로젝트를 생성해야 합니다. Firebase 프로젝트는 사용자 인증, 데이터 저장 등 웹 서비스에서 필요한 기능들을 관리하는 기본 단위입니다.

1. 작업 방법
   - Firebase 콘솔[https://console.firebase.google.com]에 접속합니다.
   - "프로젝트 만들기" 버튼을 클릭합니다.



   - 프로젝트 정보를 입력합니다.[프로젝트 이름 : 예) my-gpt-app]





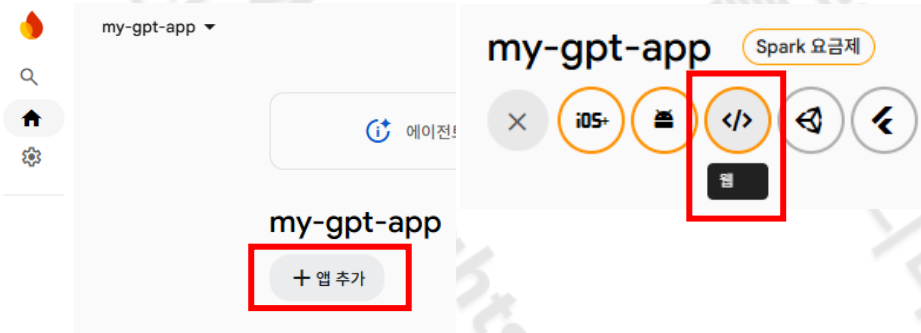
Firebase 로그인/회원가입 구현

웹 앱 등록하기

Firebase 프로젝트를 생성한 후에는 웹페이지에서 Firebase 기능을 사용할 수 있도록 웹 애플리케이션을 Firebase에 등록해야 합니다. 이 과정에서는 Firebase와 웹 페이지를 연결하기 위한 설정 정보를 발급받게 됩니다.

1. 작업 방법

- Firebase 프로젝트 생성 후, 상단 메뉴에서 웹 아이콘(</>)을 클릭합니다.



- 웹 앱 정보를 입력합니다.
  - 앱 이름 : 자유롭게 입력 (예: frontend-app)
  - Firebase Hosting 설정 : 체크하지 않아도 됨
- 등록을 완료하면 Firebase SDK 설정 코드가 화면에 표시됩니다.

© PMI. All rights reserved. 무단 배포 및 재판매 금지 / 무단 복제 및 배포 금지

Firebase 로그인/회원가입 구현

2 Firebase SDK 추가

◉ npm 사용 ○ <script> 태그 사용

이미 npm 및 모듈 번들러(예: Webpack 또는 Rollup)를 사용 중인 경우 다음 명령어를 실행하면 최신 SDK를 설치할 수 있습니다(자세히 알아보기).

```
$ npm install firebase
```

그런 다음 Firebase를 초기화하여 사용하려는 제품의 SDK를 사용하세요.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyBq-8A[illegible]8o",
  authDomain: "my-gpt-[illegible]",
  projectId: "my-gpt-a[illegible]",
  storageBucket: "my-g[illegible]ge.app",
  messagingSenderId: "[illegible]",
  appId: "1:4208020316[illegible]f8b",
  measurementId: "G-ZD[illegible]"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

- apiKey: Firebase 서비스를 사용하기 위한 인증 키
- authDomain : 로그인(인증)에 사용되는 도메인
- projectId : Firebase 프로젝트의 고유 이름
- storageBucket : 알림(메시지) 기능에 사용되는 ID
- appId : 웹 애플리케이션의 고유 식별 ID
- measurementId : 사용자 활동 분석(Analytics)을 위한 ID

firebase-config.js 파일 작성하기

웹 앱 등록 후 표시되는 Firebase 설정 정보를 이용하여 웹페이지와 Firebase를 연결하는 설정 파일을 작성합니다.

1. 작업 방법

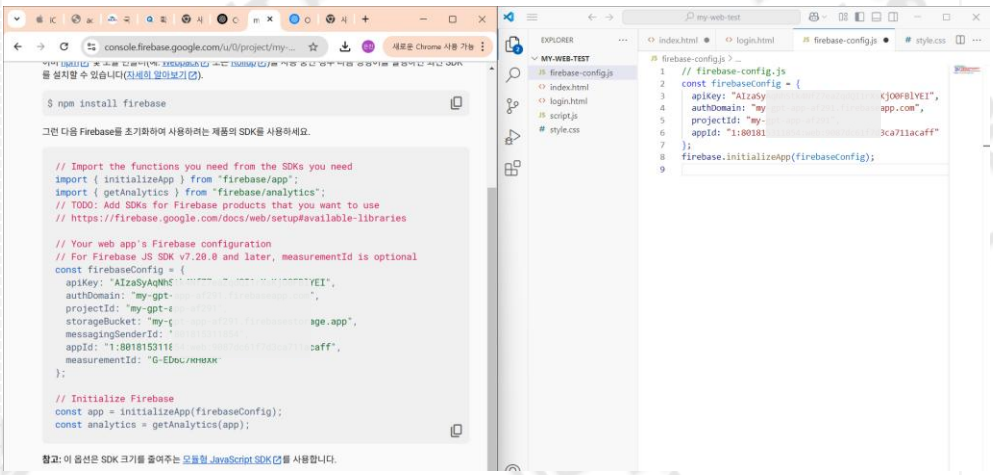
- Firebase에서 제공된 설정 코드(firebaseConfig)를 복사합니다.

Firebase 로그인/회원가입 구현

- VS Code에서 firebase-config.js 파일에 복사한 코드 추가한다.

```
const firebaseConfig = {
  apiKey: "복사한 키",
  authDomain: "xxx.firebaseapp.com",
  projectId: "xxx",
  appId: "xxx"
};

firebase.initializeApp(firebaseConfig);
```

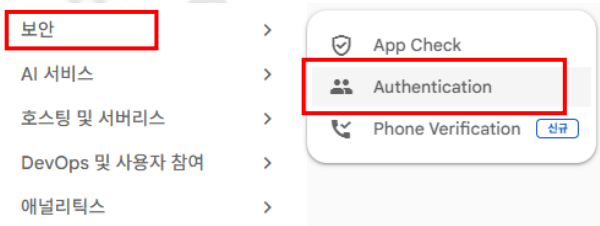


Firebase 인증(Authentication) 활성화하기

Firebase에서 회원가입과 로그인 기능을 사용하기 위해서는 인증(Authentication) 기능을 활성화해야 합니다. 이 과정에서는 이메일과 비밀번호를 이용한 로그인 방식을 설정합니다.

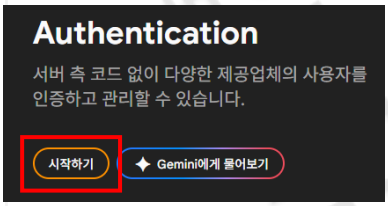
1. 작업 방법

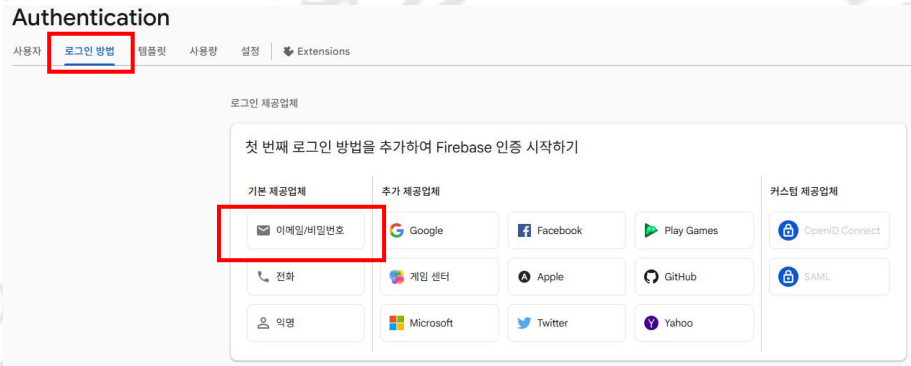
- Firebase 콘솔 왼쪽 메뉴에서 보안 - Authentication을 클릭합니다.

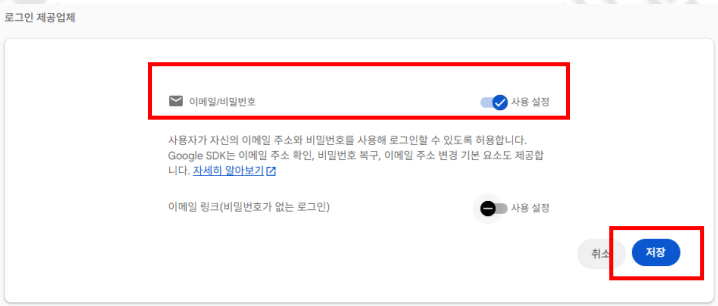


Firebase 로그인/회원가입 구현

- "시작하기" 버튼을 클릭합니다.
- 상단 메뉴에서 "로그인 방법" 탭으로 이동합니다.
- "이메일/비밀번호" 항목을 선택합니다.
- 사용 설정을 ON으로 변경한 후 저장합니다.







- Authentication : 사용자 회원가입 및 로그인 기능을 제공하는 Firebase 서비스
- 이메일/비밀번호 로그인 방식 : 가장 기본적인 사용자 인증 방법

© PNU: All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- 로그인 및 회원가입 기능 구현하기

이번 단계에서는 Firebase를 활용하여 웹페이지에서 회원가입과 로그인 기능을 직접 구현합니다. 앞에서 Firebase 설정과 인증 기능을 활성화했기 때문에, 이제 login.html에서 사용자 인증 기능을 실제로 동작 시킬 수 있습니다.

login.html 파일 작성하기

login.html은 사용자가 회원가입과 로그인을 수행하는 페이지입니다. 이 페이지에는 이메일과 비밀번호를 입력하는 입력 창, 회원가입 버튼, 로그인 버튼이 포함됩니다. 또한 Firebase에서 제공하는 인증 기능을 사용하기 위해 Firebase 관련 스크립트와 설정 파일도 함께 연결해야 합니다. 즉, 이 페이지는 단순히 입력창만 있는 화면이 아니라 사용자 입력, Firebase 인증 처리, 결과 반환의 흐름이 실제로 이루어지는 핵심 페이지입니다.

1. 작업 방법
   - VS Code 왼쪽 탐색기에서 login.html 파일을 생성합니다.
   - 파일을 클릭한 뒤, 기존 내용이 있다면 모두 삭제합니다.
   - 아래 코드를 작성합니다.

- 코드 작성

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>로그인</title>

  <!-- Firebase SDK -->
  <script src="https://www.gstatic.com/firebasejs/9.22.2/
              firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/9.22.2/
              firebase-auth-compat.js"></script>

  <!-- Firebase 설정 -->
  <script src="firebase-config.js"></script>
</head>
```

© PMI. All rights reserved. 무단 배포 금지합니다. 무단 복제 금지

Firebase 로그인/회원가입 구현

```
<body>
  <h2>회원가입</h2>
  <input id="signup-email" placeholder="이메일">
  <input id="signup-password" type="password"
placeholder="비밀번호">
  <button onclick="signup()">회원가입</button>

  <h2>로그인</h2>
  <input id="login-email" placeholder="이메일">
  <input id="login-password" type="password“
         placeholder="비밀번호">
  <button onclick="login()">로그인</button>

  <script>
    const auth = firebase.auth();

    function signup() {
      const email = document.getElementById('signup-email’).
                    value;
      const password = document.getElementById('signup
                       -password’).value;

      auth.createUserWithEmailAndPassword(email, password)
        .then(() => alert("회원가입 완료!"))
        .catch(error => alert(error.message));
    }

    function login() {
      const email = document.getElementById('login
                    -email').value;
      const password = document.getElementById('login
                       -password').value;

      auth.signInWithEmailAndPassword(email, password)
        .then(() => {
          alert("로그인 성공!");
          location.href = "page2.html"; // 로그인 후 이동
        })
        .catch(error => alert(error.message));
    }
  </script>

</body>
</html>
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- 코드 설명

1) Firebase 연결 관련

- firebase-app-compat.js : Firebase 기본 기능 사용
- firebase-auth-compat.js : 회원가입 및 로그인 기능 사용
- firebase-config.js : Firebase 프로젝트와 웹페이지 연결 설정

2) HTML 기본 구조

- <!DOCTYPE html> : HTML5 문서 형식 선언
- <html lang="ko"> : 한국어 웹페이지 설정
- <meta charset="UTF-8"> : 한글 깨짐 방지
- <title>로그인</title> : 브라우저 탭에 표시되는 제목

3) 회원가입 영역

- <h2>회원가입</h2> : 회원가입 제목 표시
- <input id="signup-email"> : 이메일 입력창
- <input id="signup-password" type="password"> : 비밀번호 입력창 (숨김 처리)
- <button onclick="signup()">회원가입</button> : 클릭 시 회원가입 함수 실행

4) 로그인 영역

- <h2>로그인</h2> : 로그인 제목 표시
- <input id="login-email"> : 로그인 이메일 입력창
- <input id="login-password" type="password"> : 로그인 비밀번호 입력창
- <button onclick="login()">로그인</button> : 클릭 시 로그인 함수 실행

5) JavaScript (기능 구현)

① Firebase 인증 객체

- const auth = firebase.auth(); : Firebase 인증 기능 사용 준비

② 회원가입 함수

- function signup() : 회원가입 기능 정의

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Firebase 로그인/회원가입 구현

- document.getElementById('signup-email').value : 입력한 이메일 가져오기
- document.getElementById('signup-password').value : 입력한 비밀번호 가져오기
- auth.createUserWithEmailAndPassword(email, password) : Firebase에 계정 생성
- .then(() => alert("회원가입 완료!")) : 성공 시 알림 표시
- .catch(error => alert(error.message)) : 실패 시 오류 메시지 표시

③ 로그인 함수

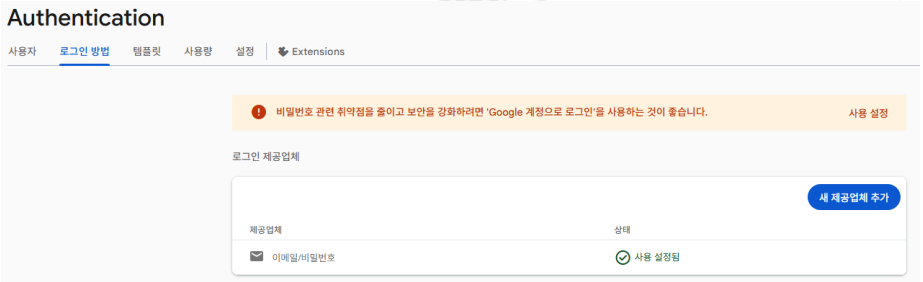
- function login() : 로그인 기능 정의
- auth.signInWithEmailAndPassword(email, password) : Firebase로 로그인 시도
- .then(() => { ... }) : 로그인 성공 시 실행
- alert("로그인 성공!") : 로그인 성공 메시지 출력
- location.href = "page2.html" : 페이지 이동
- .catch(error => alert(error.message)) : 로그인 실패 시 오류 메시지 표시

- 코드 실행 결과
  - Vs code 오른쪽 하단에 Go Live 클릭

  - Firebase 인증 설정 확인

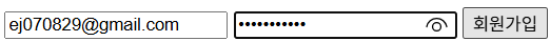
Firebase 콘솔의 Authentication - 로그인 방법 메뉴에서 "이메일/비밀번호" 방식이 사용 설정된 상태인지 확인합니다. 이 설정이 활성화되어 있어야 웹 페이지에서 회원가입과 로그인 기능을 사용할 수 있습니다

Firebase 로그인/회원가입 구현

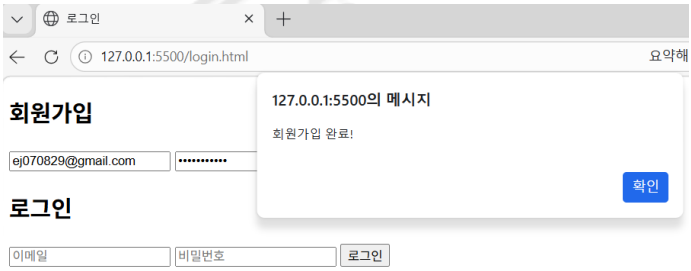


- 웹페이지에서 회원가입 실행

웹페이지에서 이메일과 비밀번호를 입력하고 회원가입 버튼을 클릭합니다.



회원가입이 정상적으로 처리되면 다음과 같은 알림창이 표시됩니다.



- Firebase 사용자 생성 확인

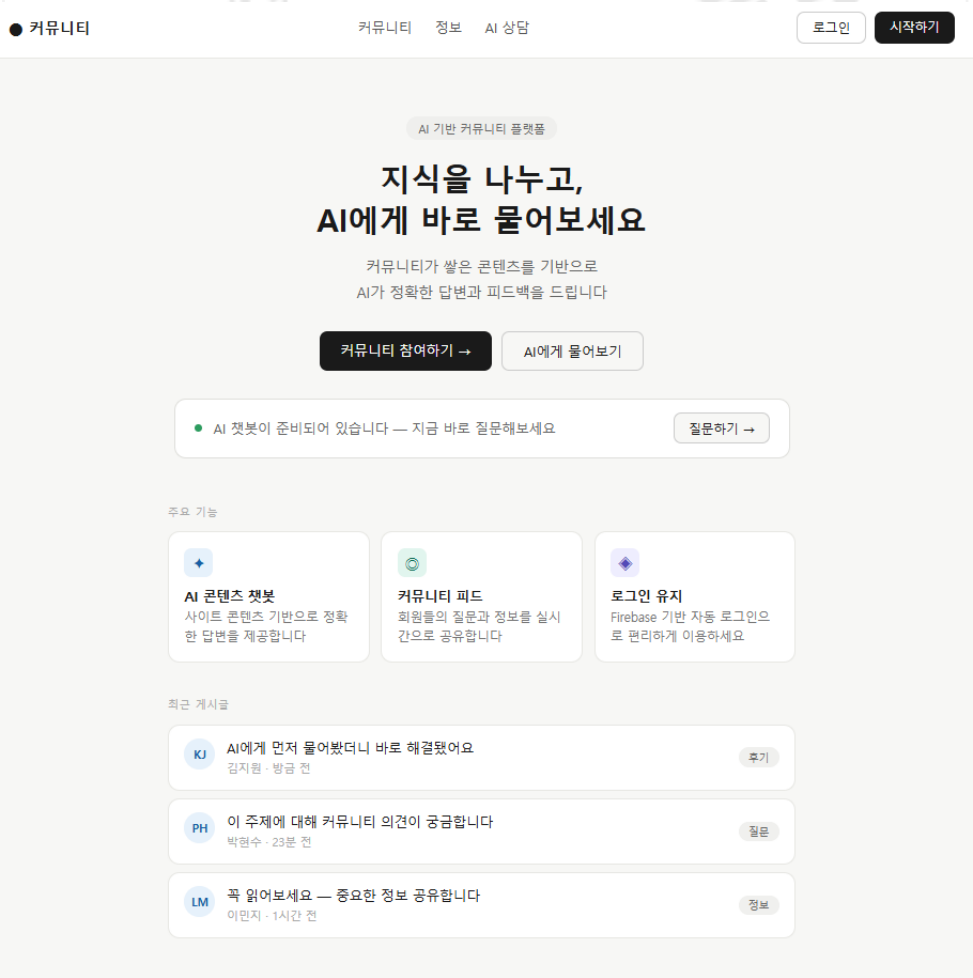
회원가입이 완료된 후, 다시 Firebase 콘솔로 이동하여 Authentication - 사용자 탭을 확인합니다. 입력한 이메일이 사용자 목록에 추가되어 있다면 회원가입이 정상적으로 완료된 것입니다.

Firebase 로그인/회원가입 구현

[AI를 활용한 실습]

1. Index.html 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.



© PNU. All rights reserved. 무단 복제 및 배포 금지입니다.

로그인 상태 유지 및 로그 아웃 기능 구현

앞 단계에서는 Firebase Authentication을 활용하여 회원가입과 로그인 기능을 구현하였다. 이를 통해 사용자가 이메일과 비밀번호를 이용해 웹페이지에 로그인할 수 있게 되었다. 하지만 실제 웹서비스에서는 단순히 로그인만 되는 것으로는 충분하지 않다. 사용자가 한 번 로그인한 뒤 페이지를 새로 고치거나 다시 접속했을 때도 로그인 상태가 유지되어야 하며, 필요할 경우 직접 로그아웃할 수 있는 기능도 함께 제공되어야 한다.

이번 단계에서는 Firebase에서 제공하는 인증 상태 확인 기능을 활용하여, 현재 사용자가 로그인된 상태인지 확인하고 그 결과에 따라 화면을 다르게 동작하도록 구현한다. 또한 로그아웃 기능을 추가하여 사용자가 서비스를 안전하게 종료할 수 있도록 구성한다.

즉, 앞에서 구현한 회원가입과 로그인 기능이 사용자 인증의 시작 단계였다면, 이번 단계는 로그인 이후의 상태를 관리하는 기능을 완성하는 과정이라고 볼 수 있다. 이 기능을 적용하면 웹페이지는 단순한 로그인 실습 화면을 넘어, 실제 서비스와 유사한 사용자 인증 흐름을 갖추게 된다.

- 로그인 상태 유지가 필요한 이유

  사용자가 페이지를 새로고침해도 다시 로그인할 필요가 없도록 하기 위함.

  현재 로그인된 사용자인지 확인하여 맞는 화면을 보여주기 위함.

  실제 웹서비스와 같은 인증 흐름을 구현하기 위함.

- Firebase에서 인증 상태 확인하기

  Firebase는 현재 로그인 상태를 확인할 수 있는 기능을 제공한다. 이를 위해 onAuthStateChanged() 함수를 사용하면 사용자의 로그인 여부를 실시간으로 확인할 수 있다. 이 함수를 활용하면 로그인 상태일 때는 메인 페이지로 이동시키고, 로그아웃 상태일 때는 로그인 화면을 보여주는 방식으로 웹페이지를 구성할 수 있다

- 로그아웃 기능 추가하기

  로그인 상태가 유지되는 기능을 구현했다면, 반대로 사용자가 원할 때 현재 인증 상태를 해제할 수 있는 로그아웃 기능도 필요하다.

로그인 상태 유지 및 로그 아웃 기능 구현

- 회원 전용 페이지 작성하기

앞 단계에서 로그인 기능을 구현하였다면, 로그인에 성공한 사용자를 위한 회원 전용 페이지가 필요하다. 이 페이지에서는 현재 로그인한 사용자의 정보를 확인하고, 로그아웃 기능을 제공한다

이번 단계에서는 회원 전용 페이지를 구성하기 위해 다음과 같은 파일을 새로 생성한다.

- page2.html : 회원 전용 화면 구성

- page2.js : 로그인 상태 확인 및 로그아웃 기능 구현

- page2.css : 페이지 스타일 적용

이와 같이 HTML, JavaScript, CSS 파일을 분리하여 구성하면 기능과 디자인을 효율적으로 관리할 수 있다

이제 page2.html 파일부터 작성해보자.

- 코드 작성

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>회원 전용 페이지</title>

  <!-- 스타일 적용 -->
  <link rel="stylesheet" href="page2.css">
  <!-- Firebase SDK -->
  <script
src="https://www.gstatic.com/firebasejs/9.22.2/firebase-
app-compat.js"></script>
  <script
src="https://www.gstatic.com/firebasejs/9.22.2/firebase-
auth-compat.js"></script>
  <!-- Firebase 설정 -->
  <script src="firebase-config.js"></script>
</head>

<body>
```

로그인 상태 유지 및 로그 아웃 기능 구현

```
  <h1>회원 전용 페이지</h1>
  <!-- 사용자 정보 출력 -->
  <p id="welcome-msg">로딩 중...</p>
  <!-- 로그아웃 버튼 -->
  <button onclick="logout()">로그아웃</button>

  <!-- JS 파일 연결 -->
  <script src="page2.js"></script>
</body>
</html>
```

- 코드 설명

1) Firebase 연결 관련

- firebase-app-compat.js : Firebase 기본 기능 사용
- firebase-auth-compat.js : 로그인/로그아웃 인증 기능 사용
- firebase-config.js : Firebase 프로젝트와 웹페이지 연결 설정

2) HTML 기본 구조

- <title>회원 전용 페이지</title> : 브라우저 탭 제목 설정
- <h1> : 페이지 제목 표시
- <p id="welcome-msg"> : 로그인한 사용자 정보를 출력하는 영역
- "로딩 중..." : JavaScript 실행 전 기본 안내 문구

3) 사용자 정보 출력 영역

- id="welcome-msg" : JavaScript에서 해당 요소를 찾아 로그인한 사용자 이메일을 표시

4) 로그아웃 버튼

- <button onclick="logout()"> : 클릭 시 page2.js 파일에 정의된 logout() 함수 실행

5) script 위치 (중요)

① <script src="page2.js"></script>를 body 하단에 위치

- Firebase 초기화 이후 JavaScript가 실행되도록 하기 위함

© PMI. All rights reserved. 무단 복제, 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

- page2.js 파일 작성하기

  회원 전용 페이지에서 로그인 상태를 확인하고, 사용자 정보를 출력하며, 로그아웃 기능을 구현하기 위해 page2.js 파일을 작성한다.

  - 파일 생성

  VS Code에서 page2.js 파일을 생성합니다.

  - 코드 작성

```
const auth = firebase.auth();

// 로그인 상태 확인
auth.onAuthStateChanged(user => {
  if (user) {
    document.getElementById('welcome-msg').textContent =
      user.email + ' 님, 환영합니다!';
  } else {
    alert('로그인이 필요합니다.');
    location.href = 'login.html';
  }
});

// 로그아웃 기능
function logout() {
  auth.signOut().then(() => {
    alert('로그아웃 되었습니다.');
    location.href = 'index.html';
  });
}
```

  - 코드 설명

  1) Firebase 인증 객체 생성

    - const auth = firebase.auth( ); : Firebase의 인증 기능을 사용하기 위한 객체 생성

  2) 로그인 상태 확인

    - auth.onAuthStateChanged(user => { }) : 현재 사용자의 로그인 상태를 확인하는 함수

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

로그인된 상태일 경우

- user 값이 존재하면 로그인 상태
- user.email을 이용하여 사용자 이메일 확인 가능

```
document.getElementById('welcome-msg').textContent =
  user.email + ' 님, 환영합니다!';
```

로그인되지 않은 상태일 경우

- user 값이 존재하면 로그인 상태
- user.email을 이용하여 사용자 이메일 확인 가능

```
alert('로그인이 필요합니다.');
location.href = 'login.html';
```

3) 로그아웃 기능

```
auth.signOut()
```

- auth.signOut() : Firebase에서 현재 로그인된 사용자 로그아웃

로그아웃 성공 시

```
alert('로그아웃 되었습니다.');
location.href = 'index.html';
```

- page2.css 파일 작성하기

회원 전용 페이지의 가독성과 사용자 경험을 높이기 위해 CSS를 활용하여 화면 디자인을 적용한다. 이번 단계에서는 page2.css 파일을 생성하여 페이지의 전체 스타일을 구성한다.

- 파일 생성

VS Code에서 page2.css 파일을 생성합니다.

- 코드 작성

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

```
body {
  font-family: '맑은 고딕', sans-serif;
  background-color: #f0f8ff;
  padding: 40px;
}

h1 {
  color: #333;
}

.welcome {
  margin-top: 10px;
  font-weight: bold;
  font-size: 18px;
  color: #007acc;
}

button {
  margin-top: 20px;
  padding: 10px 20px;
  background-color: #007acc;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
}

button:hover {
  background-color: #005fa3;
}
```

- 코드 설명

1) body { } : 전체 페이지 스타일

- font-family : 전체 글꼴 설정
- '맑은 고딕' : 한글에 적합한 기본 폰트
- background-color : 페이지 배경 색상 지정
- padding : 화면 내부 여백 설정 (콘텐츠가 가장자리와 떨어지도록)

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

2) h1{ } : 제목 스타일

- color : 제목 글자 색상 설정
- #333 : 너무 밝지도 어둡지도 않은 기본 텍스트 색상

3) .welcome { } : 사용자 메시지 스타일

- .welcome : 클래스 선택자 (사용자 정보 출력 영역에 적용)
- margin-top : 위쪽 여백
- font-weight : 글자 굵기 설정
- font-size : 글자 크기 확대
- color : 강조 색상 (파란색 계열)

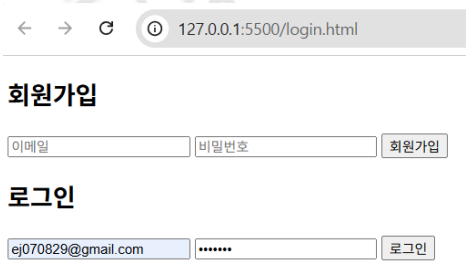
4) button { } : 버튼 스타일

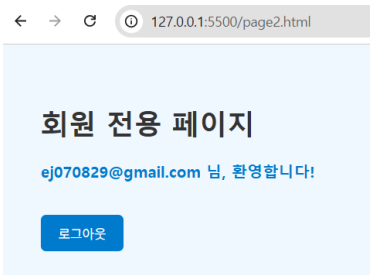
- padding : 버튼 내부 여백
- background-color : 버튼 배경 색상
- color : 버튼 글자 색상
- border-radius : 모서리를 둥글게 처리
- cursor: pointer : 마우스를 올렸을 때 클릭 가능한 형태로 변경

5) button:hover { } : 버튼 호버 효과

- :hover : 마우스를 올렸을 때 적용되는 스타일
- 버튼 색상을 어둡게 변경하여 사용자 인터랙션 효과 제공

- 코드 실행 결과





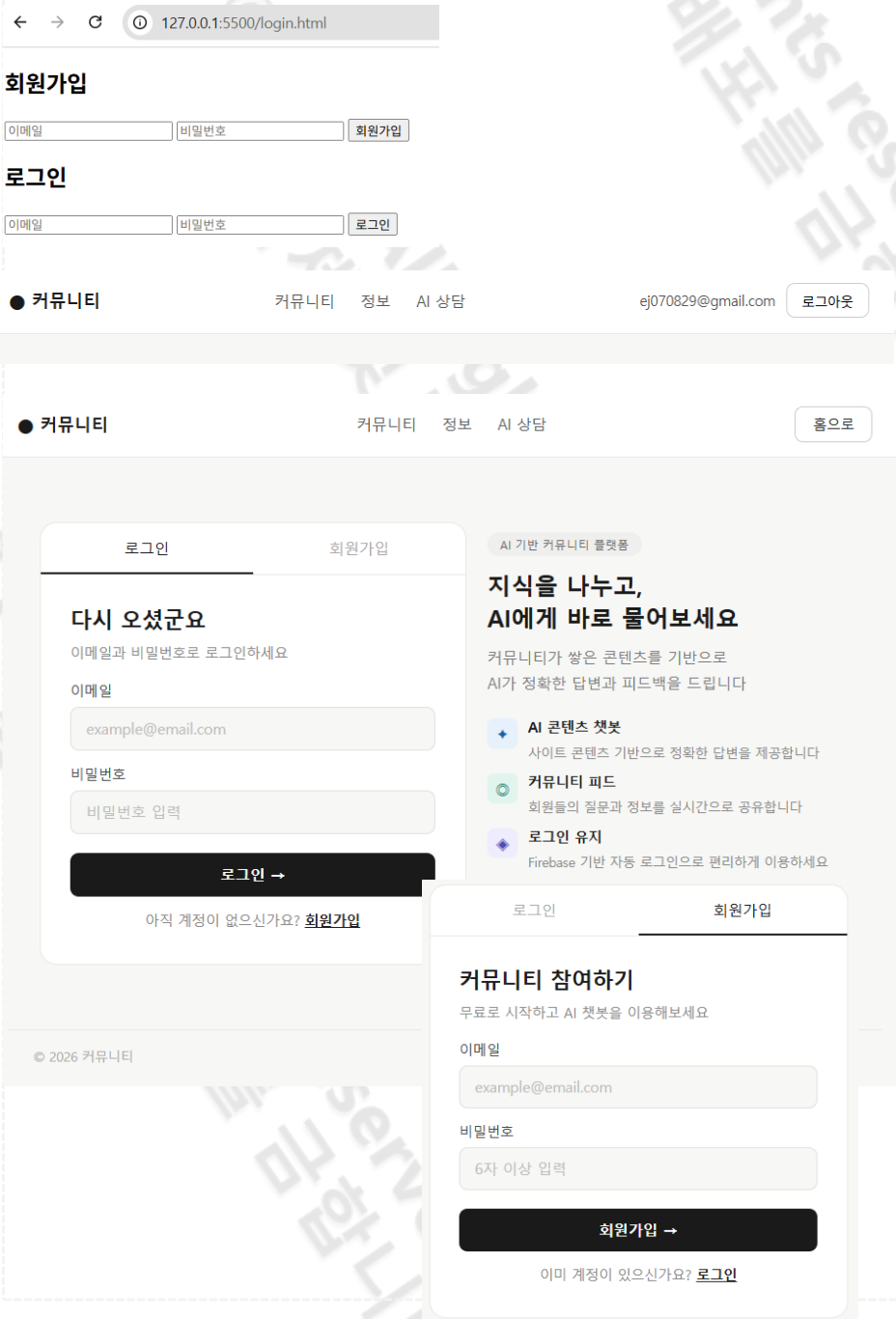
© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

로그인 상태 유지 및 로그 아웃 기능 구현

[AI를 활용한 실습]

1. login.html 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.



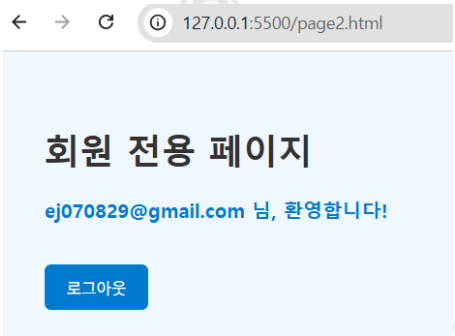
© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

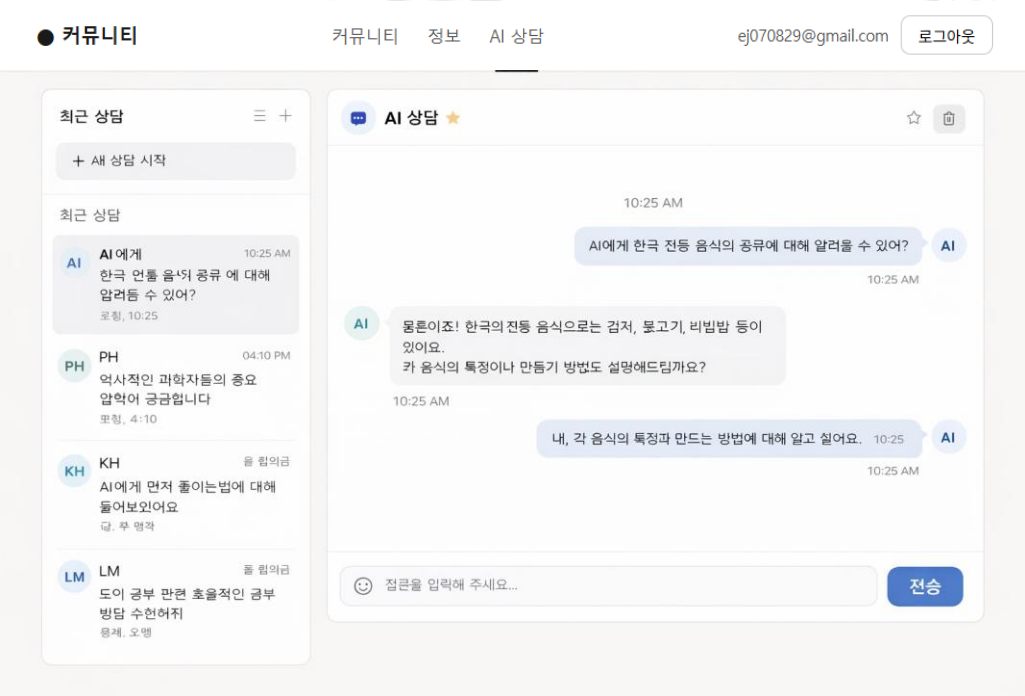
로그인 상태 유지 및 로그 아웃 기능 구현

[AI를 활용한 실습]

1. ai.html(page2.html) 화면 디자인하기

AI를 활용해서 아래와 같이 화면을 디자인해봅니다.





© PNU. All rights reserved. 무단 복제 및 배포를 금합니다.

Gemini 연동 기능 구현

이번 단계에서는 기존에 구현한 AI 상담 화면에 실제 생성형 AI 기능을 연결하여, 사용자가 입력한 질문에 대해 AI가 응답을 생성하도록 구현한다. 앞 단계까지는 사용자 인터페이스(UI)와 로그인 기능을 중심으로 웹페이지의 구조를 구성하였다면, 이번 단계에서는 외부 AI 서비스를 활용하여 웹페이지에 지능형 기능을 추가하는 과정을 학습한다.

웹페이지에서 AI 기능을 사용하기 위해서는 단순히 JavaScript만으로는 구현할 수 없으며, 외부 서버와 통신이 필요하다. 이를 위해 Python 기반의 Flask 서버를 활용하여 사용자 요청을 처리하고, Gemini API를 호출하여 응답을 받아오는 구조를 구현한다.

즉, 이번 단계는 프론트엔드(HTML, JavaScript)와 백엔드(Python)를 연결하여 하나의 완성된 AI 서비스 형태를 구현하는 과정이다.

- Gemini 연동 기능 구조 이해하기

  Gemini 기능이 포함된 웹서비스는 여러 파일이 유기적으로 연결되어 동작한다. 각 파일은 다음과 같은 역할을 수행한다.

  - HTML 파일 : 사용자 입력 및 결과 출력 화면 구성
  - JavaScript 파일 : 사용자 입력 처리 및 서버와 통신
  - Python 서버 : GPT API 호출 및 응답 반환
  - Gemini API : 생성형 AI 응답 생성

  이러한 구조를 통해 사용자의 입력이 서버를 거쳐 AI로 전달되고, 다시 결과가 웹페이지로 출력되는 흐름이 완성된다.

  다음과 같은 구조로 Gemini 연동 기능을 구성한다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Gemini 연동 기능 구현

- 구성도 설명

  1) ai.html : 사용자 입력과 AI 응답이 표시되는 화면을 구성하는 파일

  2) ai.js : 사용자 입력 처리 및 서버와 통신을 담당하는 JavaScript 파일

  3) server.py : Flask를 이용하여 Gemini API와 연결하는 백엔드 서버

  4) requirements.txt : 프로젝트 실행에 필요한 Python 라이브러리 목록

  5) css/ai.css : AI 상담 화면의 디자인을 담당하는 스타일 파일

- Gemini 연동의 동작 과정

앞에서 살펴본 Gemini 연동 구조를 바탕으로, Gemini 연동 기능이 실제로 어떻게 동작하는지 전체 흐름을 살펴보자. 각 파일은 서로 연결되어 하나의 과정으로 동작하며, 사용자의 입력이 AI의 응답으로 이어지는 흐름을 구성한다.

- Gemini 연동 기능은 다음과 같은 순서로 진행된다.

  1) 사용자가 웹페이지에서 질문을 입력한다.

  2) JavaScript가 입력값을 Flask 서버로 전송한다.

  3) 서버는 해당 데이터를 Gemini API에 전달한다.

  4) Gemini가 질문에 대한 응답을 생성한다.

  5) 서버는 응답을 받아 다시 웹페이지로 전달한다.

  6) JavaScript가 응답을 화면에 출력한다.

- Gemini API 키 발급하기

Gemini 기능을 웹페이지에서 사용하기 위해서는 Google에서 제공하는 Gemini API 키가 필요하다. API 키는 서버가 사용자의 요청을 Gemini에 전달할 때, 해당 요청이 정상적인 사용자로부터 발생했는지를 확인하는 인증 수단이다.

- API 키 발급 방법

  1) Google의 https://aistudio.google.com/api-keys 사이트에 접속한다.

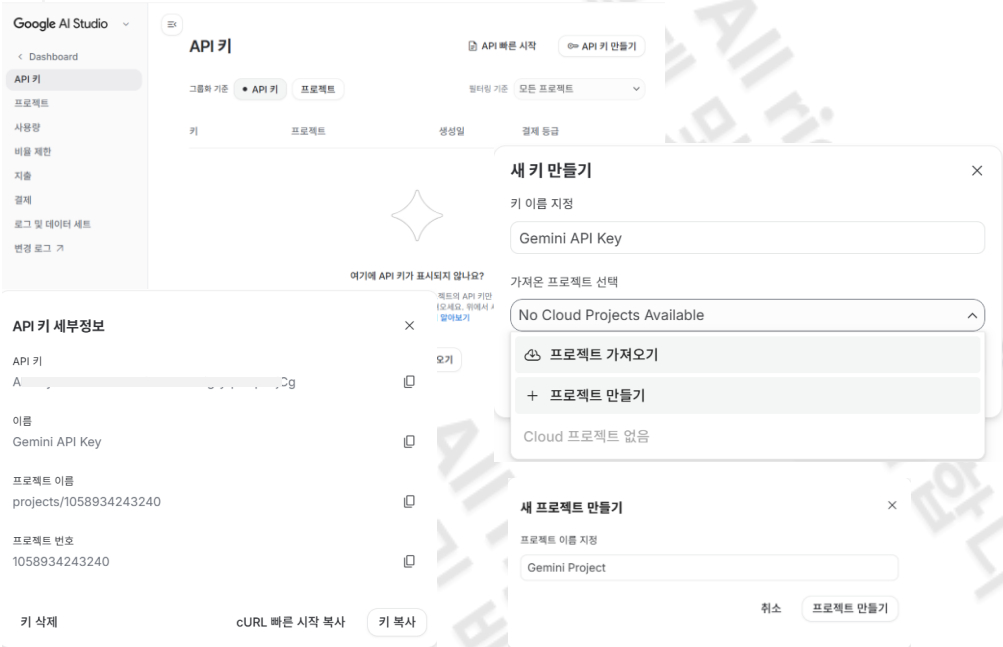
  2) Google 계정으로 로그인한 후, API 키 관리 페이지로 이동한다.

  3) 새 API 키(Create API key)를 생성한다.

  4) 발급받은 키는 .env 파일에 안전하게 저장한다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Gemini 연동 기능 구현



- API 키 사용 방법

  발급받은 API 키는 서버 코드에서 다음과 같이 사용한다.

  - Python 코드에 직접 입력하여 사용할 수 있다.
  - 또는 .env 파일과 같은 환경 변수 파일에 저장하여 불러올 수 있다.

  예를 들어, 서버 코드에서는 다음과 같은 형태로 API 키를 설정할 수 있다.

```
from google import genai
client = genai.Client(api_key="YOUR_API_KEY")
```

- API 키와 서버의 역할

  웹페이지에서는 보안 문제로 인해 Gemini API를 직접 호출하지 않고, 반드시 서버를 통해 요청을 전달해야 한다.

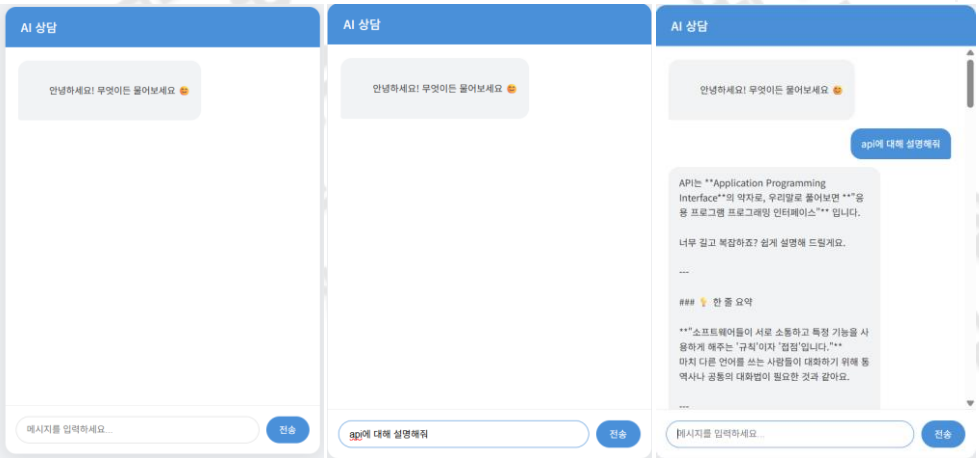
  - 웹페이지(JavaScript)는 사용자 입력만 서버로 전달
  - 서버(Python)는 API 키를 사용하여 Gemini 호출
  - 결과만 다시 웹페이지로 전달

Gemini 연동 기능 구현

- Gemini 연동 AI 상담 화면 구현

이번 단계에서는 사용자가 질문을 입력하면 Gemini API를 통해 실시간으로 답변을 받아오는 AI 상담 화면을 구현한다.

앞에서 만든 HTML, CSS, JavaScript 화면에 Python Flask 서버를 연결하여, 웹페이지에서 생성형 AI 기능을 직접 사용할 수 있도록 구성한다.



먼저, 사용자가 자유롭게 질문을 입력할 수 있는 채팅 화면을 구성한다. 사용자가 메시지를 입력하고 전송 버튼을 누르면, JavaScript가 입력된 내용을 Flask 서버로 전달한다. 서버는 전달받은 질문을 Gemini API에 전송하고, Gemini는 질문의 의미를 분석하여 적절한 답변을 생성한다. 이후 Flask 서버는 생성된 응답을 다시 웹페이지로 전달하며, JavaScript는 이를 받아 채팅창에 출력한다. 이러한 흐름을 통해 사용자는 웹페이지 안에서 실제 AI와 대화하는 것처럼 자연스럽게 질문하고 답변을 받을 수 있다.

- 동작 과정

1) 사용자가 질문을 입력한다.

2) JavaScript가 입력 내용을 Flask 서버로 전달한다.

3) Flask 서버가 Gemini API에 질문을 보낸다.

4) Gemini가 질문에 대한 답변을 생성한다.

5) 서버가 응답을 받아 웹페이지로 전달한다.

6) JavaScript가 AI 답변을 채팅창에 표시한다.

Gemini 연동 기능 구현

다음은 AI 상담 화면을 구성하는 ai.html, ai.css, ai.js 파일이다. ai.html은 채팅창, 입력창, 전송 버튼 등 화면의 기본 구조를 만들고, ai.css는 채팅창의 크기, 말풍선 모양, 버튼 디자인 등을 설정하여 사용자가 보기 편한 화면으로 꾸며준다. ai.js는 사용자가 입력한 질문을 처리하고, Flask 서버와 통신하여 Gemini의 응답을 받아 채팅창에 출력하는 기능을 담당한다. 즉, 세 파일은 각각 구조, 디자인, 기능을 맡아 서로 연결되며 하나의 AI 상담 화면을 완성한다.

- ai.html 파일 작성하기

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>AI 상담</title>
  <link rel="stylesheet" href="css/ai.css">
</head>
<body>
  <div class="chat-container">
    <div class="chat-header">
      <h1>AI 상담</h1>
    </div>
    <div class="chat-box" id="chatBox">
      <div class="message ai">
        안녕하세요! 무엇이든 물어보세요 😊
      </div>
    </div>
    <div class="chat-input-area">
      <input
        type="text"
        id="userInput"
        placeholder="메시지를 입력하세요..."
        onkeydown="if(event.key==='Enter') sendMessage()"
      />
      <button onclick="sendMessage()">전송</button>
    </div>
  </div>
  <script src="ai.js"></script>
</body>
</html>
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Gemini 연동 기능 구현

- 코드 설명
  - chat-box : 사용자와 AI의 대화 내용이 표시되는 핵심 영역으로, JavaScript가 새로운 메시지를 추가하는 공간이다.
  - id="chatBox" : JavaScript가 채팅창에 쉽게 접근할 수 있도록 지정한 식별자이다.
  - input 태그 : 사용자가 질문을 입력하는 영역으로, id="userInput"을 통해 JavaScript가 입력값을 가져온다.
  - Placeholder : 입력창에 안내 문구를 표시하여 사용자가 기능을 쉽게 이해할 수 있도록 한다.
  - onkeydown : Enter 키를 눌렀을 때도 메시지가 전송되도록 설정한 이벤트이다.
  - Button : 전송 버튼으로, 클릭 시 sendMessage() 함수가 실행된다.
  - <script src="ai.js"></script> : JavaScript 파일을 연결하여 입력 처리, 서버 통신, 응답 출력 기능이 동작하도록 한다.

- ai.css 파일 작성하기

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  font-family: 'Noto Sans KR', sans-serif;
  background-color: #f0f2f5;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.chat-container {
  width: 480px;
  height: 680px;
  background: #ffffff;
  border-radius: 16px;
  box-shadow: 0 4px 24px rgba(0, 0, 0, 0.1);
  display: flex;
```

```
    flex-direction: column;
    overflow: hidden;
  }
  .chat-header {
    background: #4A90D9;
    color: white;
    padding: 18px 24px;
  }
  .chat-header h1 {
    font-size: 18px;
    font-weight: 500;
  }
  .chat-box {
    flex: 1;
    padding: 20px;
    overflow-y: auto;
    display: flex;
    flex-direction: column;
    gap: 12px;
  }
  .message {
    max-width: 75%;
    padding: 12px 16px;
    border-radius: 16px;
    font-size: 14px;
    line-height: 1.6;
    white-space: pre-wrap;
    word-break: break-word;
  }
  .message.ai {
    background: #f1f3f5;
    color: #333;
    align-self: flex-start;
    border-bottom-left-radius: 4px;
  }
  .message.user {
    background: #4A90D9;
    color: white;
    align-self: flex-end;
    border-bottom-right-radius: 4px;
  }
  .chat-input-area {
    display: flex;
    padding: 16px;
    gap: 10px;
```

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

```
    border-top: 1px solid #eee;
  }
  .chat-input-area input {
    flex: 1;
    padding: 10px 16px;
    border: 1px solid #ddd;
    border-radius: 24px;
    font-size: 14px;
    outline: none;
  }
  .chat-input-area input:focus {
    border-color: #4A90D9;
  }
  .chat-input-area button {
    padding: 10px 20px;
    background: #4A90D9;
    color: white;
    border: none;
    border-radius: 24px;
    font-size: 14px;
    cursor: pointer;
  }
  .chat-input-area button:hover {
    background: #357ABD;
  }
```

- ai.js 파일 작성하기

```
  async function sendMessage() {
    const input = document.getElementById("userInput");
    const chatBox = document.getElementById("chatBox");
    const message = input.value.trim();
    if (!message) return;
    appendMessage("user", message);
    input.value = "";
    const loadingEl = appendMessage("ai", "입력 중...");
    try {
      const response = await
  fetch("http://127.0.0.1:5000/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: message })
      });
      const data = await response.json();
```

```
        loadingEl.remove();
        appendMessage("ai", data.reply || "오류가 발생했습니다.");

      } catch (error) {
        loadingEl.remove();
        appendMessage("ai", "서버에 연결할 수 없습니다.
server.py가 실행 중인지 확인하세요.");
      }
      chatBox.scrollTop = chatBox.scrollHeight;
    }
    function appendMessage(role, text) {
      const chatBox = document.getElementById("chatBox");
      const div = document.createElement("div");
      div.className = `message ${role}`;
      div.textContent = text;
      chatBox.appendChild(div);
      chatBox.scrollTop = chatBox.scrollHeight;
      return div;
    }
```

- 코드 설명

1) 메시지 입력값 가져오기

```
const input = document.getElementById("userInput");
const chatBox = document.getElementById("chatBox");
const message = input.value.trim();
```

- 입력창(userInput)과 채팅창(chatBox) 요소를 가져온다.
- 사용자가 입력한 메시지의 앞뒤 공백을 제거하여 정리한다.

2) 빈 메시지 전송 방지

```
if (!message) return;
```

- 입력값이 없으면 함수를 종료하여 빈 메시지가 전송되지 않도록 한다.

3) 사용자 메시지 출력

```
appendMessage("user", message);
input.value = "";
```

- 사용자가 입력한 내용을 먼저 채팅창에 표시한다.
- 메시지 전송 후 입력창을 비워 다음 질문을 입력할 수 있도록 한다.

Gemini 연동 기능 구현

4) 로딩 메시지 표시

- const loadingEl = appendMessage("ai", "입력 중..."); : AI가 응답을 생성하는 동안 "입력 중..." 메시지를 출력하여 처리 상태를 사용자에게 알려준다.

5) 서버로 질문 전송 : `const response`

- fetch()를 사용해 Flask 서버의 /chat 주소로 요청을 보낸다.
- POST 방식으로 사용자의 질문을 JSON 형식으로 전달한다

6) 서버 응답 처리 : `const data = await response.json();`

- 서버에서 받은 응답을 JSON 형태로 변환한다.
- POST 방식 → 데이터를 서버로 전송
- 로딩 메시지를 제거한 뒤, Gemini가 생성한 답변을 채팅창에 출력한다.

7) 오류 처리 : `catch (error) { … }`

- 서버 연결에 실패하거나 오류가 발생하면 안내 메시지를 출력한다.

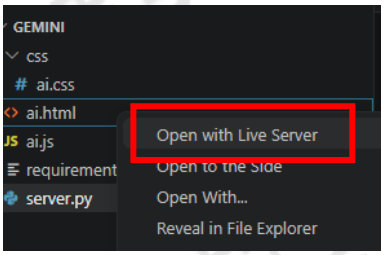
8) 채팅창 자동 스크롤 : `chatBox.scrollTop = chatBox.scrollHeight;`

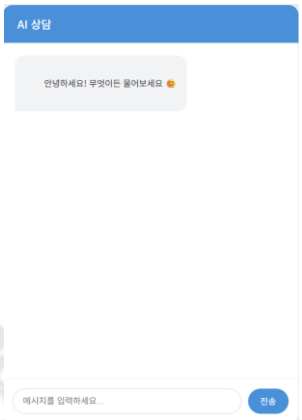
- 새로운 메시지가 추가될 때마다 채팅창이 자동으로 아래로 이동하여 최신 메시지가 보이도록 한다.

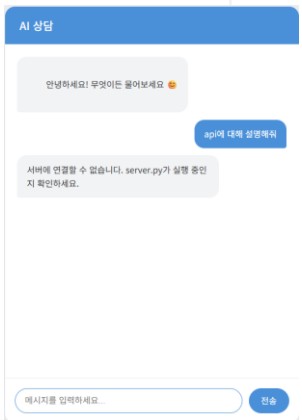
9) 메시지 출력 함수 : `function appendMessage(role, text) { … }`

- 사용자와 AI의 메시지를 채팅창에 추가하는 공통 함수이다.
- 역할(user, ai)에 따라 말풍선 스타일을 구분하여 출력한다.

- 코드 실행 결과







© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

Gemini 연동 기능 구현

앞에서 작성한 ai.html, ai.css, ai.js 파일을 모두 저장한 뒤, AI 상담 화면이 정상적으로 표시되는지 실행하여 확인해본다. Visual Studio Code에서 ai.html 파일을 우클릭한 후 Live Server를 실행하면 브라우저에서 화면을 확인할 수 있다. 실행 결과, 상단에는 'AI 상담' 제목이 표시되고, 가운데에는 AI의 기본 인사말이 출력된 채팅창이 나타난다. 하단에는 입력창과 전송 버튼이 배치되어 실제 채팅 서비스와 유사한 화면이 구성된다.

또한 사용자가 메시지를 입력하고 전송 버튼을 누르면, 입력한 질문이 채팅창에 표시되며, JavaScript를 통해 화면이 자연스럽게 갱신되는 것을 확인할 수 있다. 아직 이 단계에서는 Flask 서버와 Gemini API를 연결하지 않았기 때문에 실제 AI 응답은 생성되지 않지만, 코드 실행 결과를 통해 화면 구조와 사용자 입력 처리 기능이 정상적으로 동작하는지 미리 점검할 수 있다.

다음 단계에서는 사용자 질문을 받아 Gemini API에 전달하고, 생성된 응답을 다시 웹페이지로 반환하는 server.py 파일을 작성해본다. 앞에서 ai.html, ai.css, ai.js를 통해 AI 상담 화면과 사용자 입력 처리 기능을 구현했다면, 이제는 Python 기반의 Flask 서버를 연결하여 실제로 AI가 답변을 생성할 수 있도록 구성해야 한다. server.py는 웹페이지와 Gemini API를 이어 주는 파일로, 사용자가 입력한 질문을 서버에서 받아 처리하고 그 결과를 다시 화면에 전달하는 핵심 역할을 담당한다.

- server.py 파일 작성하기

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from google import genai
from dotenv import load_dotenv
import os
load_dotenv()
app = Flask(__name__)
CORS(app)
api_key = "자신이 발급 받은 키 추가"
if not api_key:
    raise ValueError("GEMINI_API_KEY가 설정되지
않았습니다. .env 파일을 확인하세요.")
client = genai.Client(api_key=api_key)
@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
```

```
        if not data:
            return jsonify({"error": "요청 데이터가
                                 없습니다."}), 400
        user_input = data.get("message", "").strip()
        if not user_input:
            return jsonify({"error": "메시지가 비어
                                 있습니다."}), 400
        try:
            response = client.models.generate_content(
                model="gemini-2.5-flash",
                contents=user_input
            )
            reply_text = response.text if response.text
            else "응답을 생성하지 못했습니다."
            return jsonify({"reply": reply_text})
        except Exception as e:
            return jsonify({"error": f"Gemini 호출 중 오류
                                 발생: {str(e)}"}), 500

    if __name__ == "__main__":
        app.run(debug=True)
```

- 코드 설명

1) 필요한 라이브러리 불러오기

- Flask : 웹 서버를 생성하는 라이브러리이다.
- request : 웹페이지에서 전달된 데이터를 받는 데 사용한다.
- jsonify : 서버의 응답을 JSON 형식으로 변환한다.
- CORS : 브라우저와 서버 간의 통신을 허용한다.
- genai : Gemini API를 호출하는 라이브러리이다.
- dotenv, os : 환경 변수나 API 키를 관리할 때 사용된다.

2) Flask 서버 생성 및 CORS 설정

```
app = Flask(__name__)
CORS(app)
```

- Flask 서버를 생성하고, 브라우저에서 서버로 요청을 보낼 수 있도록 CORS를 설정한다.

Gemini 연동 기능 구현

3) Flask 서버 생성 및 CORS 설정

```
api_key = "YOUR_API_KEY"
client = genai.Client(api_key=api_key)
```

- Gemini API 키를 설정하여 Gemini와 연결한다.(116페이지 참조)
- genai.Client()를 통해 Gemini를 호출할 준비를 한다.

4) 채팅 요청 처리 주소 설정

```
@app.route("/chat", methods=["POST"])
def chat():
```

- /chat 주소로 들어오는 POST 요청을 처리하는 함수이다.
- JavaScript에서 사용자가 입력한 질문을 이 주소로 전달한다.

5) 사용자 질문 데이터 받기

```
data = request.get_json()
user_input = data.get("message", "").strip()
```

- JavaScript가 보낸 JSON 데이터를 받아온다.
- 사용자가 입력한 질문 내용을 추출하고 공백을 제거한다.

6) 입력값 검사 : `if not data:, if not user_input:`

- 요청 데이터가 없거나 메시지가 비어 있으면 오류를 반환하여 잘못된 요청을 방지한다.

7) Gemini API 호출 : `response = client.models.generate_content( ... )`

- 사용자의 질문을 Gemini API에 전달하여 답변을 생성한다.
- gemini-2.5-flash 모델을 사용해 빠르게 응답하도록 설정하였다.

8) Gemini 응답 반환 : `reply_text =`

- Gemini가 생성한 답변을 가져와 JSON 형식으로 웹페이지에 전달한다.
- 응답이 없을 경우 안내 문구를 반환한다.

© PMI. All rights reserved. 무단 복제 및 배포를 금합니다.

9) 서버 실행

```
if __name__ == "__main__":
    app.run(debug=True)
```

- server.py 파일을 직접 실행할 때 Flask 서버를 시작한다.
- debug=True는 개발 중 오류를 쉽게 확인할 수 있도록 도와준다.

다음 단계에서는 앞에서 작성한 server.py 파일이 정상적으로 실행될 수 있도록, 프로젝트에 필요한 Python 라이브러리를 설치해본다. Python 프로그램은 필요한 기능에 따라 여러 외부 라이브러리를 사용하게 되는데, Flask 서버 실행, 브라우저와의 통신, Gemini API 호출 등을 위해 관련 라이브러리가 미리 설치되어 있어야 한다. 이때 필요한 라이브러리 목록을 한 번에 정리해두는 파일이 requirements.txt이다. 이 파일을 사용하면 프로젝트를 실행할 때 필요한 패키지를 쉽게 설치할 수 있고, 다른 환경에서도 동일한 실행 환경을 빠르게 구성할 수 있다.

- requirements.txt 파일 작성하기

```
flask
flask-cors
google-genai
python-dotenv
```

- 코드 설명
  - flask : 웹 서버를 실행하는 라이브러리이다.
  - flask-cors : 브라우저와 서버 간 통신을 허용한다.
  - google-genai : Gemini API를 호출하는 라이브러리이다.
  - python-dotenv : 환경 변수와 API 키를 관리할 때 사용된다.

- 라이브러리 설치

```
pip install -r requirements.txt
```

  - 터미널에서 위 명령어를 입력하면 필요한 라이브러리가 한 번에 설치된다.
  - 이를 통해 Gemini 연동에 필요한 실행 환경을 쉽게 준비할 수 있다.

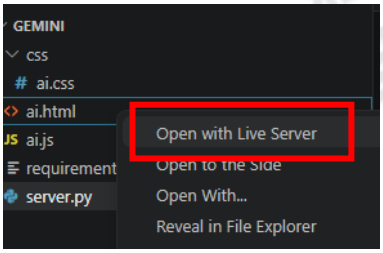
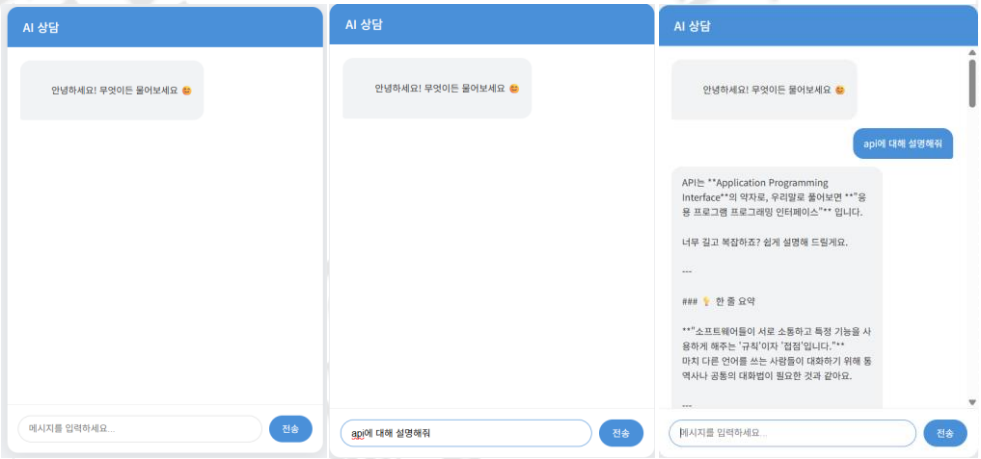
Gemini 연동 기능 구현

설치가 완료되면 아래 명령어를 입력하여 Flask 서버를 실행한다.

```
python server.py
```

정상적으로 실행되면 터미널에 로컬 서버 주소가 표시되며, 이는 Gemini와 연결할 준비가 완료되었다는 의미이다. 이후 Visual Studio Code에서 ai.html 파일을 우클릭한 뒤 Live Server를 실행하거나 브라우저에서 직접 열어 AI 상담 화면에 접속한다. 사용자가 채팅창에 질문을 입력하고 전송 버튼을 누르면, JavaScript가 입력 내용을 Flask 서버로 전달하고, 서버는 Gemini API를 호출하여 생성된 답변을 다시 웹페이지에 출력한다. 이 과정을 통해 앞에서 작성한 모든 파일이 서로 연결되어 실제로 동작하는 AI 상담 서비스를 확인할 수 있다.

- 코드 실행



- Gemini 연동 구현 결과